Task activation functions

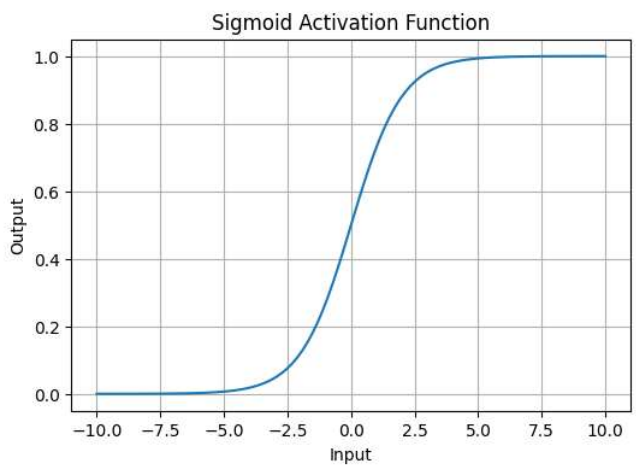
An activation function in a neural network is a mathematical function applied to the output of a neuron. It introduces non-linearity, enabling the model to learn and represent complex data patterns. Without it, even a deep neural network would behave like a simple linear regression model.

Activation functions decide whether a neuron should be activated based on the weighted sum of inputs and a bias term. They also make backpropagation possible by providing gradients for weight updates.

1. Sigmoid Function

Sigmoid / Logistic

$$f(x) = \frac{1}{1 + e^{-x}}$$



Pros:

- Output values between 0 and 1, useful for probability prediction in binary classification.
- Smooth and differentiable.

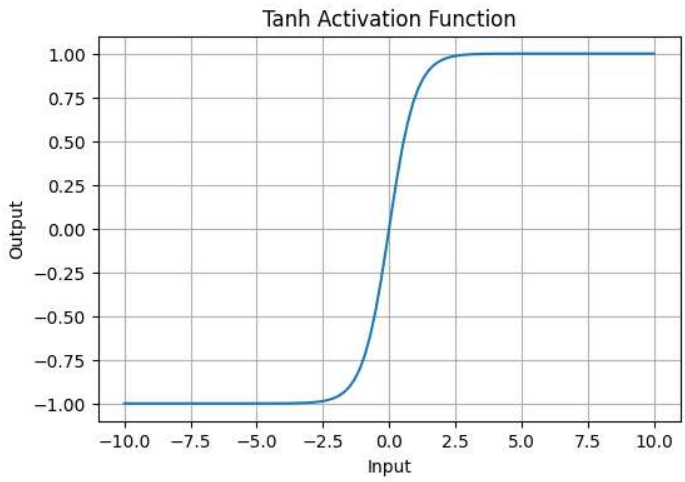
Cons:

- Vanishing gradient problem when inputs are very large or very small.
- Not zero-centered, which slows optimization.

2. Tanh Activation Function

$$Tanh$$

$$f(x) = \frac{\left(e^{x} - e^{-x}\right)}{\left(e^{x} + e^{-x}\right)}$$



Pros:

- Zero-centered output, which improves gradient updates.
- Stronger gradients compared with sigmoid.

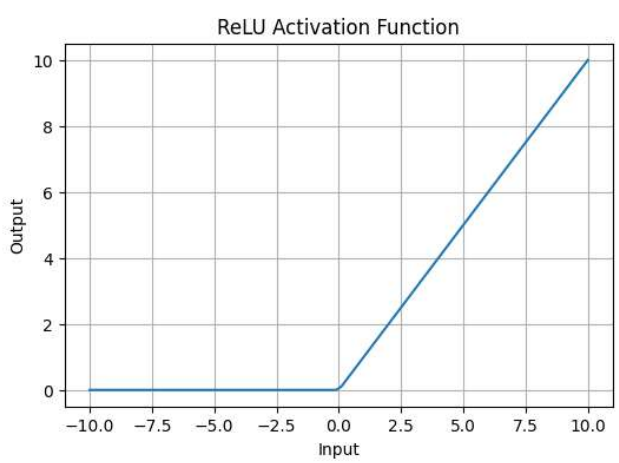
Cons:

- vanishing gradient problem when inputs are very large or very small.
- Computationally is more expensive than simple functions.

3. **ReLU** (Rectified Linear Unit) **Function**

$$ReLU$$

$$f(x) = max\ (0, x)$$



Pros:

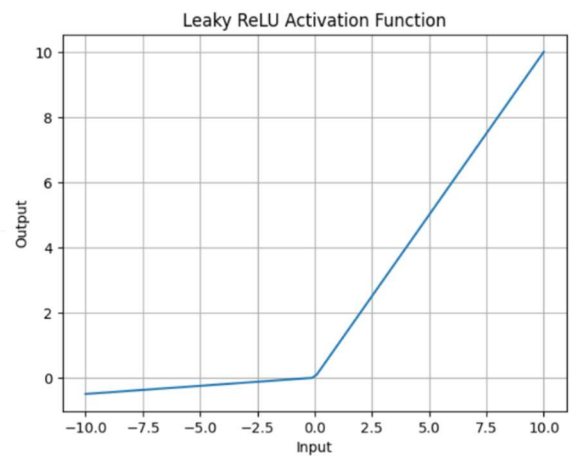
- Helps reduce the vanishing gradient problem.
- Fast training

Cons:

- Not zero-centered.

4. Leaky ReLU

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases},$$



Pros:

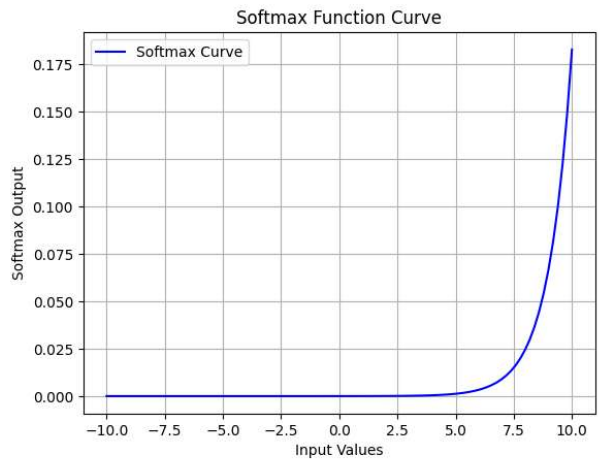
- Solves the dead neuron problem of ReLU.
- Allows a small gradient for negative inputs.

Cons:

- Choice of α parameter is not always optimal.

5. Softmax Function

$$\sigma(x_j) = \frac{e^{x_j}}{\sum_i e^{x_i}}$$



Pros:

- Converts outputs into probability distribution.
- Best for multi-class classification problems.

Cons:

- Sensitive to **large input values**.